

P.PORTO

NodeJS Express Middleware

Programação em Ambiente Web

Índice

Introdução

Express Framework

Template Engines

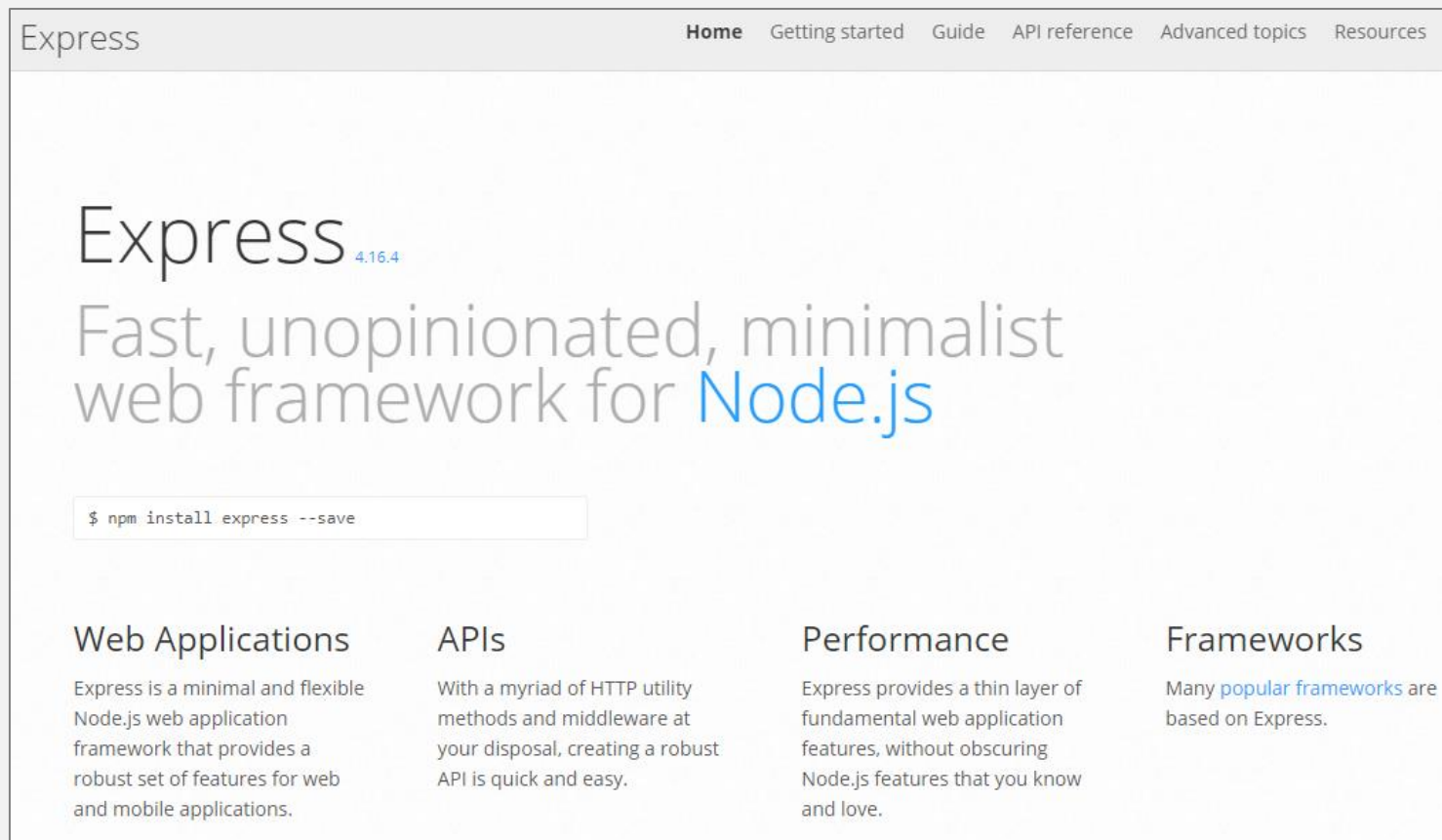
Padrão MVC

Middleware

Referências

Introdução

O que é o ExpressJS?



The screenshot shows the Express.js website homepage. At the top, there is a navigation bar with links: Home, Getting started, Guide, API reference, Advanced topics, and Resources. The main heading is 'Express' followed by the version number '4.16.4'. Below this, the tagline reads 'Fast, unopinionated, minimalist web framework for Node.js'. A code block shows the command '\$ npm install express --save'. The page is divided into four columns: 'Web Applications', 'APIs', 'Performance', and 'Frameworks', each with a brief description of Express.js capabilities.

Express

4.16.4

Fast, unopinionated, minimalist web framework for Node.js

```
$ npm install express --save
```

Web Applications	APIs	Performance	Frameworks
Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.	With a myriad of HTTP utility methods and middleware at your disposal, creating a robust API is quick and easy.	Express provides a thin layer of fundamental web application features, without obscuring Node.js features that you know and love.	Many popular frameworks are based on Express.

Fonte: <https://expressjs.com/>

Introdução

O Express é uma Framework Web NodeJS minimal e flexível que fornece um conjunto robusto de recursos para desenvolver aplicações na Web

Facilita o rápido desenvolvimento de aplicações para a Web baseados em NodeJS



Introdução

Foi criado por TJ Holowaychuk

Permite um rápido desenvolvimento de aplicações baseadas em NodeJS para a Web

É agnóstica do ponto vista dos *design patterns*, podendo o programador utilizar os *design pattern* que preferir



Introdução

Algumas das características nucleares da framework:

- Disponibiliza um conjunto robusto de funcionalidades para aplicações Web
- Permite definir *middleware* para lidar com pedidos HTTP
- Define uma tabela de *routing* que é utilizada para realizar diferentes ações mediante o método HTTP e o URL
- Permite apresentar dinamicamente páginas HTML com argumentos e templates



Express Framework



Express Framework

Começar um projeto com a Express Framework é semelhante a um projeto NodeJS

Existe um gerador que pode ajudar nesta tarefa

- Express generator

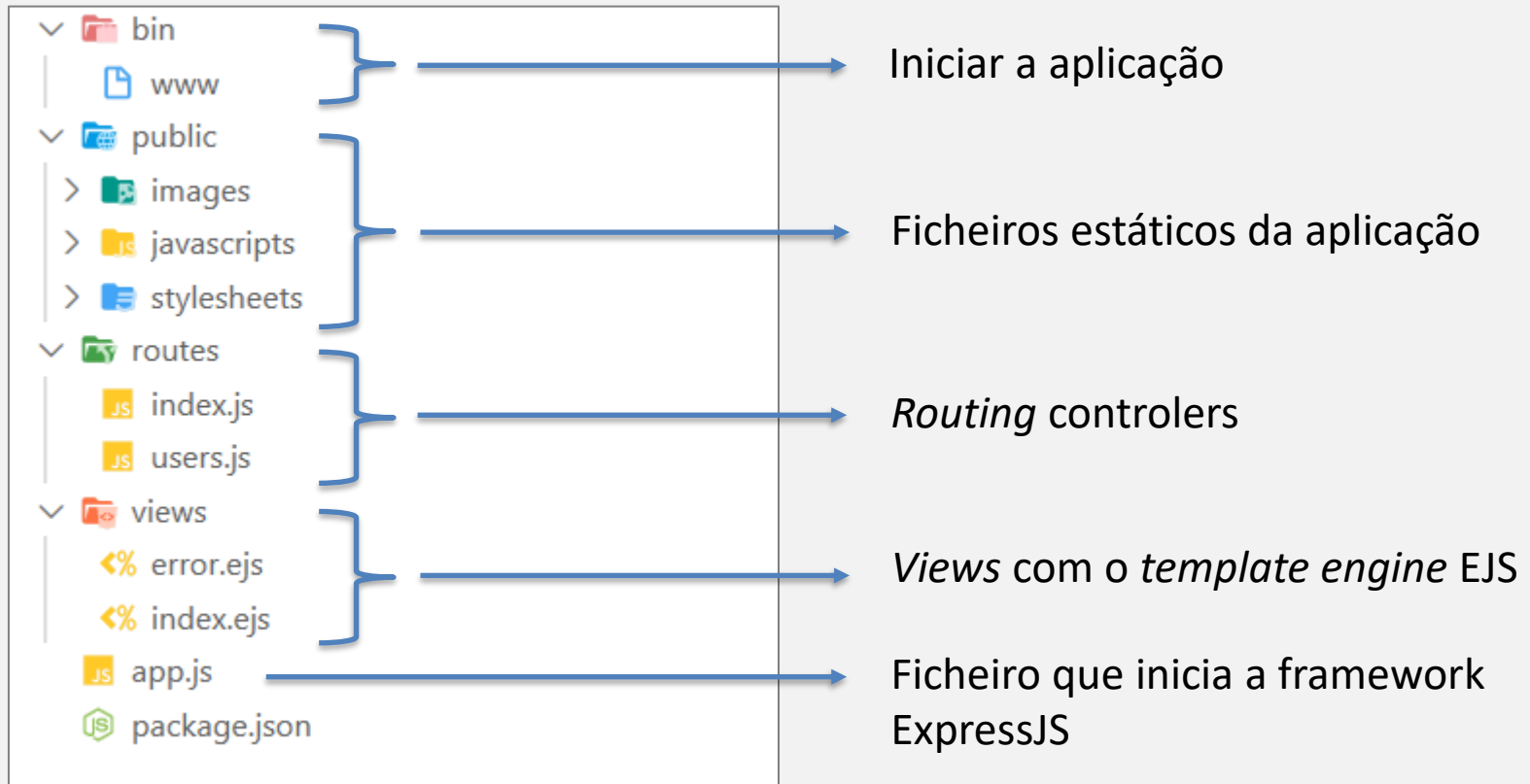
Instalar:

```
npm install -g express-generator
```

Iniciar o gerador de projetos com:

```
express
```


Express Framework



Express Framework

Instalar todos os módulos
já pré configurados com:

– npm install

Este gerador cria
automaticamente

- routing controllers para
“/users” e “/”

```
var indexRouter = require('./routes/index');
var usersRouter = require('./routes/users');

var app = express();

// view engine setup
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'ejs');

app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());
app.use(express.static(path.join(__dirname, 'public')));

app.use('/', indexRouter);
app.use('/users', usersRouter);
```

Express Framework

Iremos instalar um conjunto de módulos que irão ser úteis em tarefas e agir como middleware na framework express

- `npm install body-parser --save`
- `npm install cookie-parser --save` (já deve estar instalado)
- `npm install multer --save`

Verifique que o ficheiro `packages.json` foi atualizado com estes módulos como dependências

Express Framework

Body-parser

incluí funcionalidades para fazer o parse do body de pedidos http para o servidor.

Cookie-parser

incluí funcionalidades para fazer o parse de cookies nos pedidos http para o servidor

Multer

incluí funcionalidades para processar mutlipart http request, como o caso do envio de ficheiros para o servidor

Express Framework

As aplicações em Express utilizam uma função de callback em que os parâmetros são objetos de request (pedidos) e response (resposta)

- **Request object:** representa um pedido HTTP e possui propriedades para os pedidos query string, parâmetros, corpo, cabeçalhos HTTP «, etc.
- **Response object:** representa uma resposta HTTP que uma aplicação Express envia quando recebe um pedido HTTP.

Pode imprimir os objetos “req” e “res”, que disponibilizam muita informação relacionada com o pedido e resposta HTTP, incluindo cookies, sessões, URL, etc.

Express Framework

Request

- req.query
permite lidar com query strings
- req.params
faz o parsing dos parâmetros de routing
 - Exemplo: <http://127.0.0.1:3000/orders/123x9>
 - » a porção 123x9 do route pode ser um parâmetro
- req.body
utilizado para fazer o parsing do corpo
- req.param(nome)
devolve o valor específico do pedido do cliente
- req.path, req.host, req.ip
auto-explicativo!
- req.cookies
devolve todos os cookies do pedido

Express Framework

Response

- `res.status(código)`
define o código do status da resposta HTTP (e.g. 404)
- `res.set(campo, valor)`
é utilizado para definir os cabeçalhos HTTP
- `res.cookie(nome, valor, [opções])`
define o cookie de resposta
- `res.redirect(status, url)`
redireciona para outro URL
- `res.send(corpo)`
respostas “non-streaming” para um pedido. Define o “Content-Type” e o “Content-length”
- `res.json(corpo)`
envia um objeto ou array (comum em servidores SPA/Web)
- `res.render(view, locals, callback)`
pre-render uma página HTML

Express Framework

Application

- `app.set(nome,valor)`: define as variáveis para o Express
- `app.get(nome)`: devolve as variáveis do ambiente
- `app.engine(ext, callback)`: trabalha com o motor de templates para renderizar tipos de ficheiros
- `app.locals`: envia variáveis da aplicação para os templates
- `app.use(path, callback)`: especificações do middleware
- `app.VERB(path, callback)`: é utilizado para lidar com os campos dos pedidos verbs HTTP como middleware
- `app.route(path).VERB(callback, callback)`: responde aos verbs HTTP quando um determinado route/caminho é especificado
- `app.param(nome, callback)`: utilizado para responder quando um determinado parâmetro de routing é encontrado

Express Framework

Routing refere-se a determinar como uma aplicação irá responder quando o cliente faz um determinado pedido para um particular *endpoint*

O Express suporta os seguintes métodos de *routing*:

- checkout
- copy
- delete
- get
- head
- lock
- merge
- mkactivity

- mkcol
- move
- m-search
- notify
- options
- patch
- post

- purge
- put
- report
- search
- subscribe
- trace
- unlock
- unsubscribe

A documentação da API tem entradas somente para os métodos HTTP mais populares: `app.get()`, `app.post()`, `app.put()`, e `app.delete()`

Express Framework

Exemplo de routing

```
const express = require('express') //
const app = express()

// This responds with "Hello World" on the homepage
app.get('/', function (req, res) {
  console.log("Got a GET request for the homepage");
  res.send('Hello GET');
})

// This responds a POST request for the homepage
app.post('/', function (req, res) {
  console.log("Got a POST request for the homepage");

  res.send('Hello POST');
})

// This responds a DELETE request for the /del_user page.
app.delete('/del_user', function (req, res) {
  console.log("Got a DELETE request for /del_user");
  res.send('Hello DELETE');
})

// This responds a GET request for the /list_user page.
app.get('/list_user', function (req, res) {
  console.log("Got a GET request for /list_user");
  res.send('Page Listing');
})
```

Express Framework

Exemplo de routing

```
// This responds a GET request for abcd, abxcd, abl23cd, and so on
app.get('/ab*cd', function(req, res) {
  console.log("Got a GET request for /ab*cd");
  res.send('Page Pattern Match');
})

var server = app.listen(3000, function () {
  var host = server.address().address
  var port = server.address().port
  console.log("Example app listening at http://%s:%s", host, port)
})
```

Express Framework

Exemplo de routing

```
app.get('/users/:userId/books/:bookId', function (req, res) {  
  res.send(req.params)  
})
```

- Criar request parameters a partir do url do pedido
- Os parameters estão anotados por “:” no endereço
- Uso:

```
Route path: /users/:userId/books/:bookId  
Request URL: http://localhost:3000/users/34/books/8989  
req.params: { "userId": "34", "bookId": "8989" }
```

Express Framework

Exemplo de routing

```
app.get('/example/b', function (req, res, next) {  
  console.log('the response will be sent by the next function ...')  
  next()  
}, function (req, res) {  
  res.send('Hello from B!')  
})
```

- Podemos também encadear diferentes callbacks para a mesmo pedido ao servidor
- Teremos de chamar o método next() em todas as chamadas intermédias

Express Framework

Podemos também usar o objeto *routing* para subdividir a nossa aplicação por partes lógicas, controladores de *routing*

```
// wiki.js - Wiki route module.  
  
var express = require('express');  
var router = express.Router();  
  
// Home page route.  
router.get('/', function (req, res) {  
  res.send('Wiki home page');  
})  
  
// About page route.  
router.get('/about', function (req, res) {  
  res.send('About this wiki');  
})  
  
module.exports = router;
```

```
var wiki = require('./wiki.js');  
// ...  
app.use('/wiki', wiki);
```

Express Framework

Podemos definir um parâmetro dentro do routing para servir os nossos ficheiros estáticos como:

- CSS
- JavaScript
- Images
- ..

```
var express = require('express');
var app = express();

app.use('/public', express.static('public'));

app.get('/', function (req, res) {
  res.send('Hello World');
})

var server = app.listen(8081, function () {
  var host = server.address().address
  var port = server.address().port

  console.log("Example app listening at http://%s:%s", host, port)
})
```

- Neste caso os ficheiros ficam disponível a partir do endereço:
 - localhost:8081/public/"file"

Express Framework

Podemos usar o express para facilmente obter dados de um formulário da web

Considere o seguinte formulário com método GET:

```
<html>
  <body>
    <form action = "http://127.0.0.1:8081/process_get" method = "GET">
      First Name: <input type = "text" name = "first_name"> <br>
      Last Name: <input type = "text" name = "last_name">
      <input type = "submit" value = "Submit">
    </form>
  </body>
</html>
```


Express Framework

Podemos usar o ExpressJS para facilmente obter dados de um formulário

Podemos usar a ExpressJS para processar o formulário do lado do servidor

```
var express = require('express');
var app = express();

app.use(express.static('public'));
app.get('/index.htm', function (req, res) {
  res.sendFile( __dirname + "/" + "index.html" );
})

app.get('/process_get', function (req, res) {
  // Prepare output in JSON format
  response = {
    first_name:req.query.first_name,
    last_name:req.query.last_name
  };
  console.log(response);
  res.end(JSON.stringify(response));
})
```

```
var server = app.listen(8081, function () {
  var host = server.address().address
  var port = server.address().port

  console.log("Example app listening at http://%s:%s", host, port)
})
```

Express Framework

Considere agora a mesmo formulário com método POST:

```
<html>
  <body>

    <form action = "http://127.0.0.1:8081/process_post" method = "POST">
      First Name: <input type = "text" name = "first_name"> <br>
      Last Name: <input type = "text" name = "last_name">
      <input type = "submit" value = "Submit">
    </form>

  </body>
</html>
```

Express Framework

Podemos usar o ExpressJS para facilmente obter dados de um formulário

```
var express = require('express');
var app = express();
var bodyParser = require('body-parser');

// Create application/x-www-form-urlencoded parser
var urlencodedParser = bodyParser.urlencoded({ extended: false })

app.use(express.static('public'));
app.get('/index.htm', function (req, res) {
  res.sendFile( __dirname + "/" + "index_post.html" );
})

app.post('/process_post', urlencodedParser, function (req, res) {
  // Prepare output in JSON format
  response = {
    first_name:req.body.first_name,
    last_name:req.body.last_name
  };
  console.log(response);
  res.end(JSON.stringify(response));
})
```

```
var server = app.listen(8081, function () {
  var host = server.address().address
  var port = server.address().port

  console.log("Example app listening at http://%s:%s", host, port)
})
```

Express Framework

Verifique o facto de o formulário seja submetido via método POST requer no servidor um parse do conteúdo do request HTTP

Para facilitar a tarefa foi usado o módulo body-parser

Express Framework

Formulário multipart para lidar com submissão de imagens/vídeos ou outros ficheiros

```
<html>
  <head>
    <title>File Uploading Form</title>
  </head>

  <body>
    <h3>File Upload:</h3>
    Select a file to upload: <br />

    <form action = "http://127.0.0.1:8081/file_upload" method = "POST"
      enctype = "multipart/form-data">
      <input type="file" name="file" size="50" />
      <br />
      <input type = "submit" value = "Upload File" />
    </form>

  </body>
</html>
```

Express Framework

ExpressJS ajuda-nos também com formulários multipart, com a ajuda do módulo multer

Ideal para gerir a submissão de ficheiros

Necessário guardar os ficheiros numa pasta do servidor

Express Framework

```
var express = require('express');
var fs = require('fs');
var bodyParser = require('body-parser');
var multer = require('multer');

// Multer storage options
var storage = multer.diskStorage({
  destination: function(req, file, cb) {
    cb(null, 'tmp/');
  },
  filename: function(req, file, cb) {
    cb(null, file.originalname + '-' + Date.now() + '.pdf');
  }
});

var upload = multer({ storage: storage });

//var upload = multer({ dest: 'tmp/'})

var app = express();

app.use(express.static('public'));
app.use(bodyParser.urlencoded({ extended: false }));

app.use(upload.single('file'));

app.get('/index.htm', function (req, res) {
  res.sendFile( __dirname + "/" + "file_form.html" );
})

app.post('/file_upload', function (req, res) {
  console.log(req.file.name);
  console.log(req.file.path);
  console.log(req.file.type);
  var file = __dirname + "/" + req.file.name;
```

```
fs.readFile( req.files.file.path, function (err, data) {
  fs.writeFile(file, data, function (err) {
    if( err ) {
      console.log( err );
    } else {
      response = {
        message:'File uploaded successfully',
        filename:req.files.file.name
      };
    }
    console.log( response );
    res.end( JSON.stringify( response ) );
  });
});

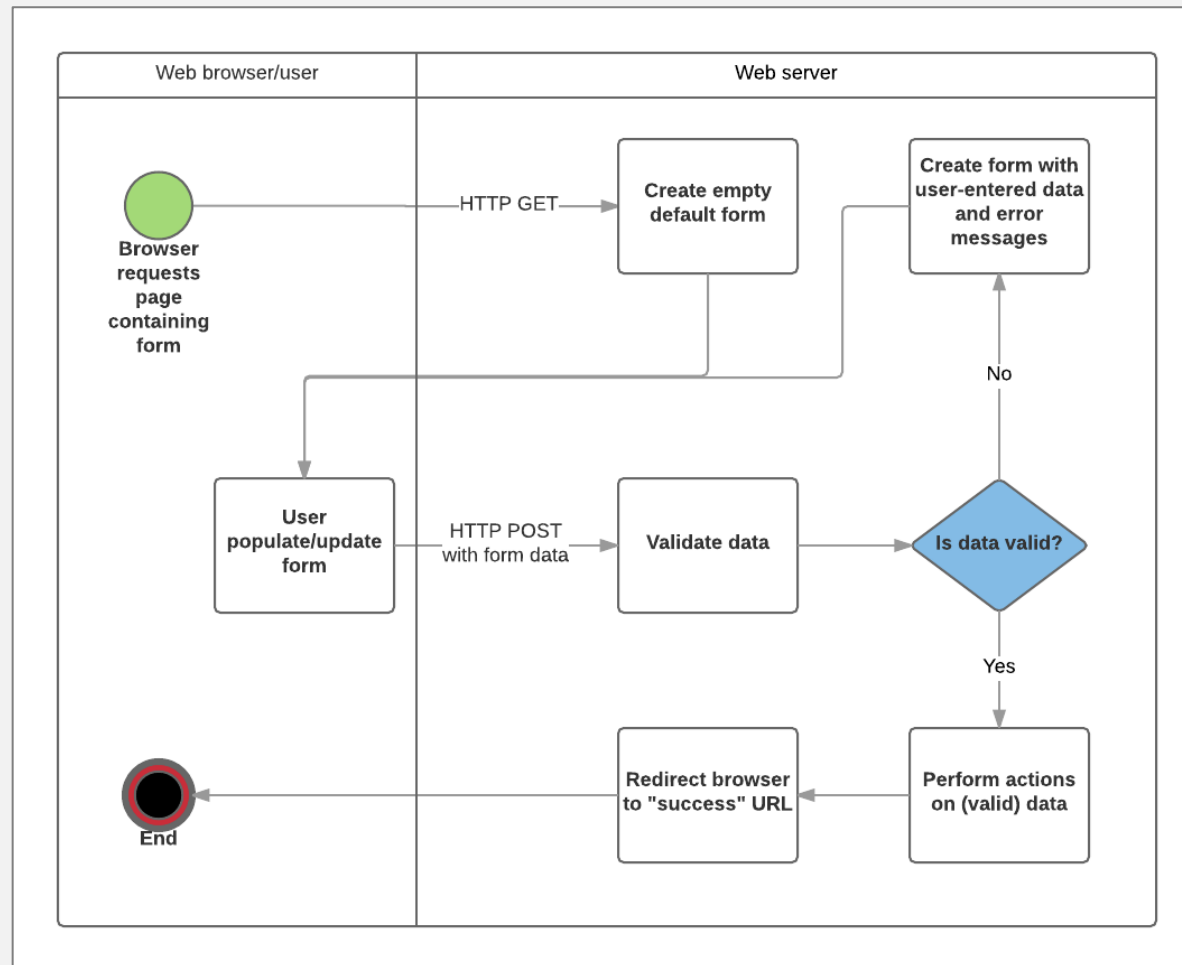
var server = app.listen(8081, function () {
  var host = server.address().address
  var port = server.address().port

  console.log("Example app listening at http://%s:%s", host, port)
})
```

Express Framework

O processamento é semelhante, necessitamos de copiar o ficheiro que foi *uploaded* para uma pasta temporária no servidor para uma nova pasta

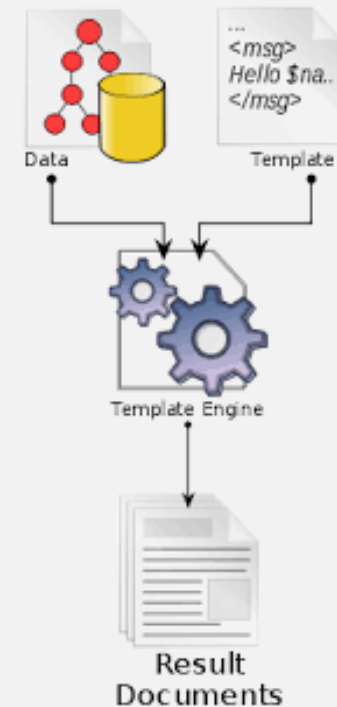
Express Framework



Fonte: <https://mdn.mozillademos.org/files/14478/Web%20server%20form%20handling.png>

Template Engine

`<%= EJS %>`



pug

Template Engine

Um template engine permite usar ficheiros estáticos de modelo estáticos a sua aplicação

Em tempo de execução, o template engine substitui as variáveis num ficheiro modelo por valores reais e transforma o modelo num ficheiro HTML e envia para o cliente

Essa abordagem facilita o design de uma página HTML e facilita a criação de páginas que se adaptam ao conteúdo

Template Engine

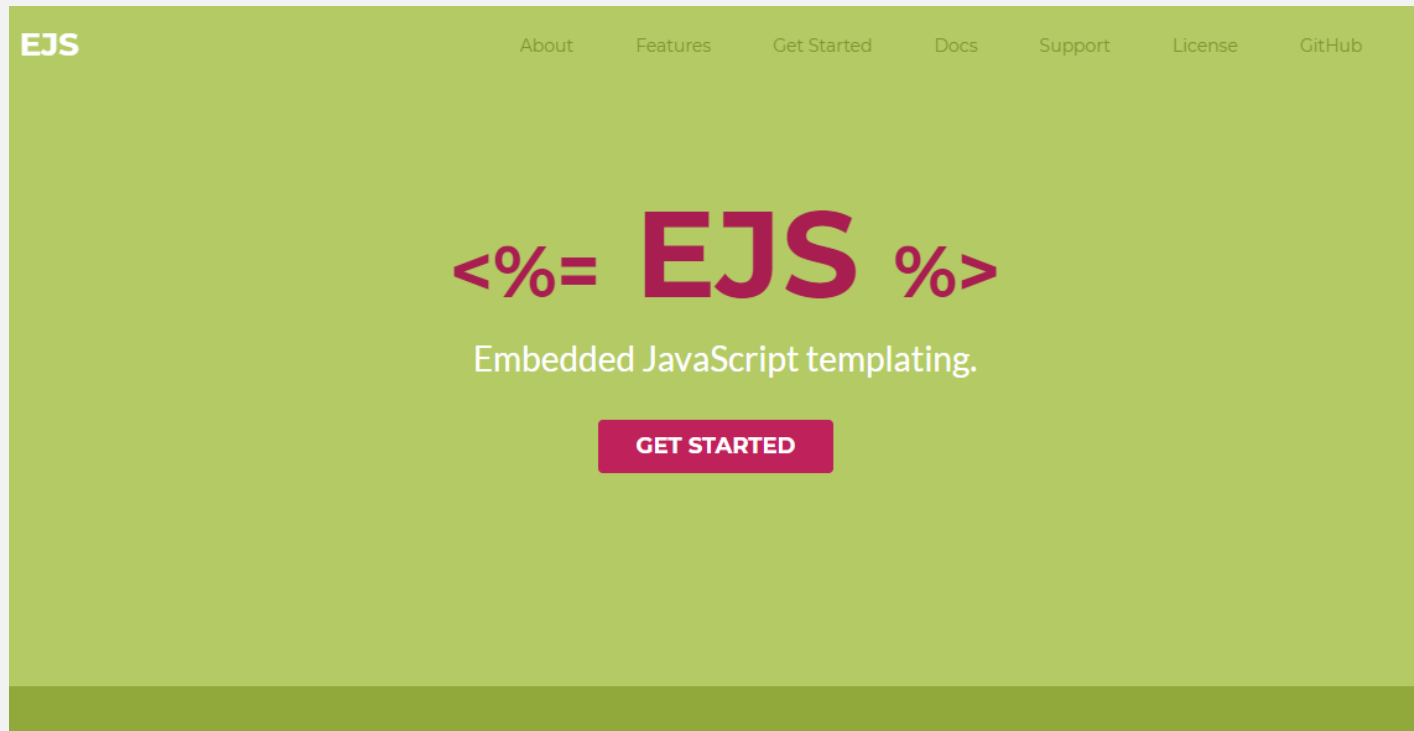
Alguns template engine populares que funcionam com o Express são:

- o Pug,
- o Moustache
- o EJS
- ...

O gerador de aplicações do express usa o Pug como padrão, mas também suporta outros.

Template Engine

EJS



Fonte: <https://ejs.co/>

Template Engine

EJS é também um template engine disponível para NodeJS

Difere pela syntax usada para criar o conteúdo dinâmico nas páginas

Permite o uso direto de código javascript usando tags específicas definidas na sua especificação

Template Engine

EJS

- Usa tags directamente no código das páginas em HTML
- Alguns exemplos de tags neste template engine
 - `<% 'Scriptlet' tag, for control-flow, no output`
 - `<%=` Outputs the value into the template (HTML escaped)
 - `<%-` Outputs the unescaped value into the template
 - `<%#` Comment tag, no execution, no output
 - `<%%` Outputs a literal '`<%`'
 - `%>` Plain ending tag
 - `-%>` Trim-mode ('newline slurp') tag, trims following newline

Template Engine

EJS

- Análise de exemplos

```
<h1>Users</h1>

<% function user(user) { %>
  <li><strong><%= user.name %></strong> is a <%= user.age %> year old <%= user.species %>.</li>
<% } %>

<ul>
  <% users.map(user) %>
</ul>
```

- Numa coleção de users executa a função user em cada um deles gerando os items numa lista em html

Template Engine

EJS

– Análise de exemplos

```
<% if (names.length) { %>
  <ul>
    <% names.forEach(function (name) { %>
      <## Notice how the single quotation mark is escaped in the output HTML
      <li foo='<%= name + "'" %>'><%= name %></li>
    <% }) %>
  </ul>
<% } %>
```

– Enquanto houverem names, para cada um criar uma entrada na lista

Template Engine

EJS

– Análise de exemplos

```
<% if (names.length) { %>
  <ul>
    <% names.forEach(function (name) { %>
      <## Notice how the single quotation mark is escaped in the output HTML
      <li foo='<%= name + "'" %>'><%= name %></li>
    <% }) %>
  </ul>
<% } %>
```

- Enquanto houverem names, para cada um criar uma entrada na lista
- Mais exemplos em <https://github.com/mde/ejs>

Template Engine

EJS

- Não suporta blocos, mas suporta includes para criar os nossos layouts

```
<%- include('header'); -%>
<h1>
|   Title
</h1>
<p>
|   My page
</p>
<%- include('footer'); -%>
```

```
<ul>
  <% users.forEach(function(user){ %>
    |   <%- include('user/show', {user: user}); %>
    <% }); %>
</ul>
```

Template Engine

EJS

– Instalar

- npm install ejs --save

– Demo

- Criar o ficheiro exemplo.ejs

```
<% if (user) { %>
<h2><%= user.name %></h2>
<% } %>
```

- Criar o ficheiro demo.js

```
var ejs = require('ejs');

ejs.renderFile("exemplo.ejs", {user:{name:'me'}}, function(err, str){
  console.log(str);
});
```

Template Engine

Demo

Template Engine

Demo: Express + EJS

- Abra o terminal
- Crie uma diretoria (e.g. myapp2)
- Mude para diretoria criada
- Na consola: `express -v ejs -css myapp2`
- Na consola: `npm install`
- Na consola: `npm start`

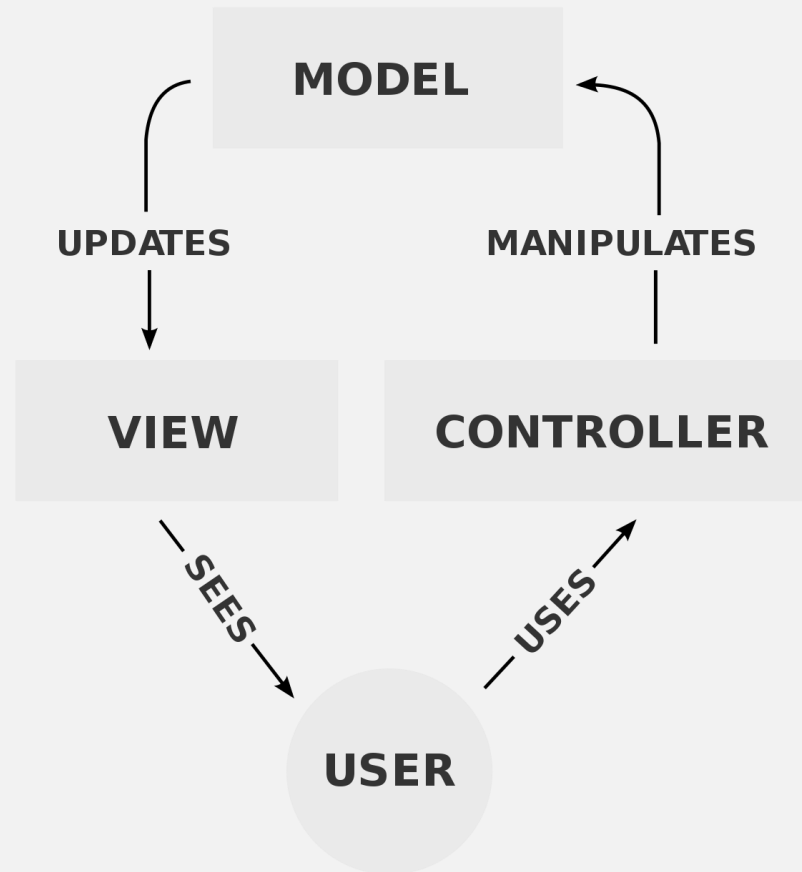
Padrão MVC

MVC – significa Model, View, Controller

Representa um padrão para separa logicamente elementos da nossa aplicação web:

- Model – modelos de dados
- View – Visualização e elementos da interface
- Controller – Elementos que controlão a view e lhe associam os dados do modelo

Padrão MVC



Padrão MVC

Deveremos no futuro organizar o código das nossas aplicações de acordo com esta estrutura

Usaremos o conceito de módulo e export de ES6 para separar cada conteúdo dentro das nossas aplicações

O generator de express já o tenta fazer ainda que de forma limitada

Padrão MVC

É possível cumprir o padrão MVC com a ExpressJS?

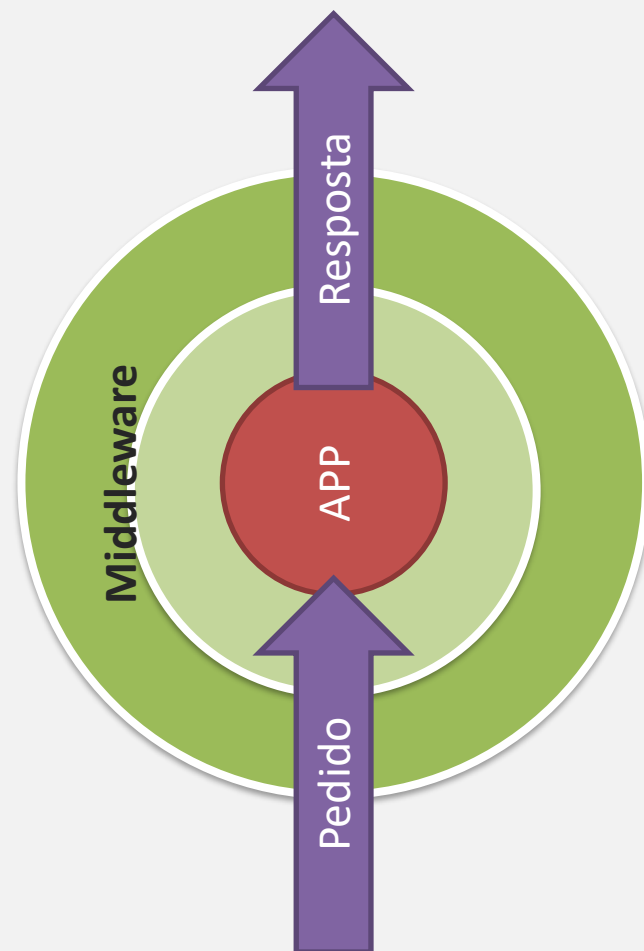
Qual o componente em falta neste momento?

Middleware

É o conjunto de funções que são invocadas pelo *ExpressJS* antes da última camada que lida com o pedido

São funções que têm acesso direto ao objeto do pedido (req) e da resposta (res) e à próxima função de *middleware* da aplicação

A próxima função de middleware é numa cadeia habitualmente denominada por uma variável de nome *next()*



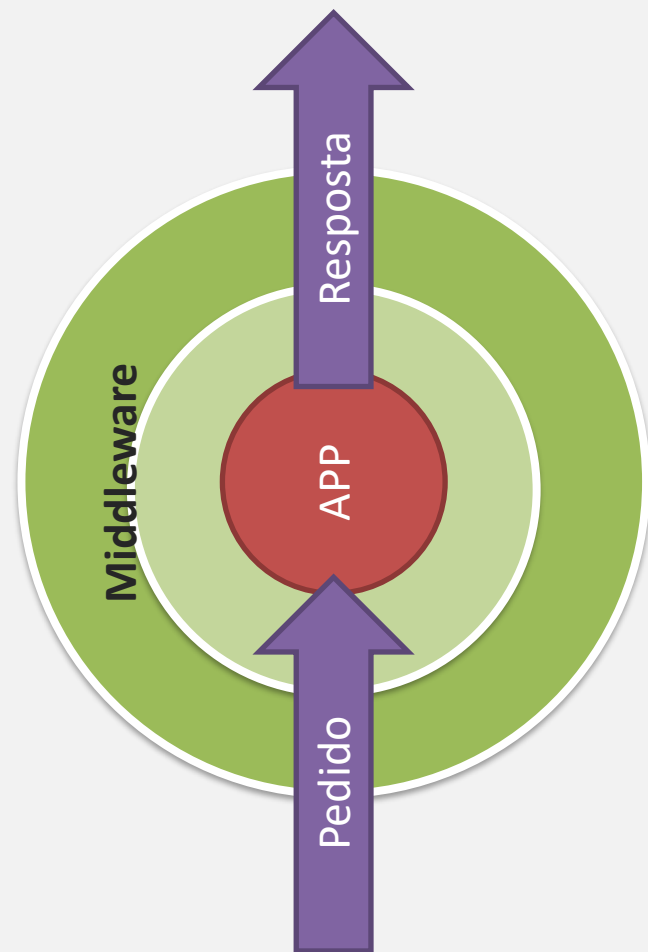
Middleware

Uma função de *middleware* básica em ExpressJS

```
function myMiddleware (req, res, next) {  
  getSomeData()  
  .then(function (data) {  
    req.somedata = data  
    next()  
  })  
}
```

Utilização

```
var app = express()  
app.use(myMiddleware)  
  
//your normal route Handlers  
app.get('/someroute', handler)  
...
```



Middleware

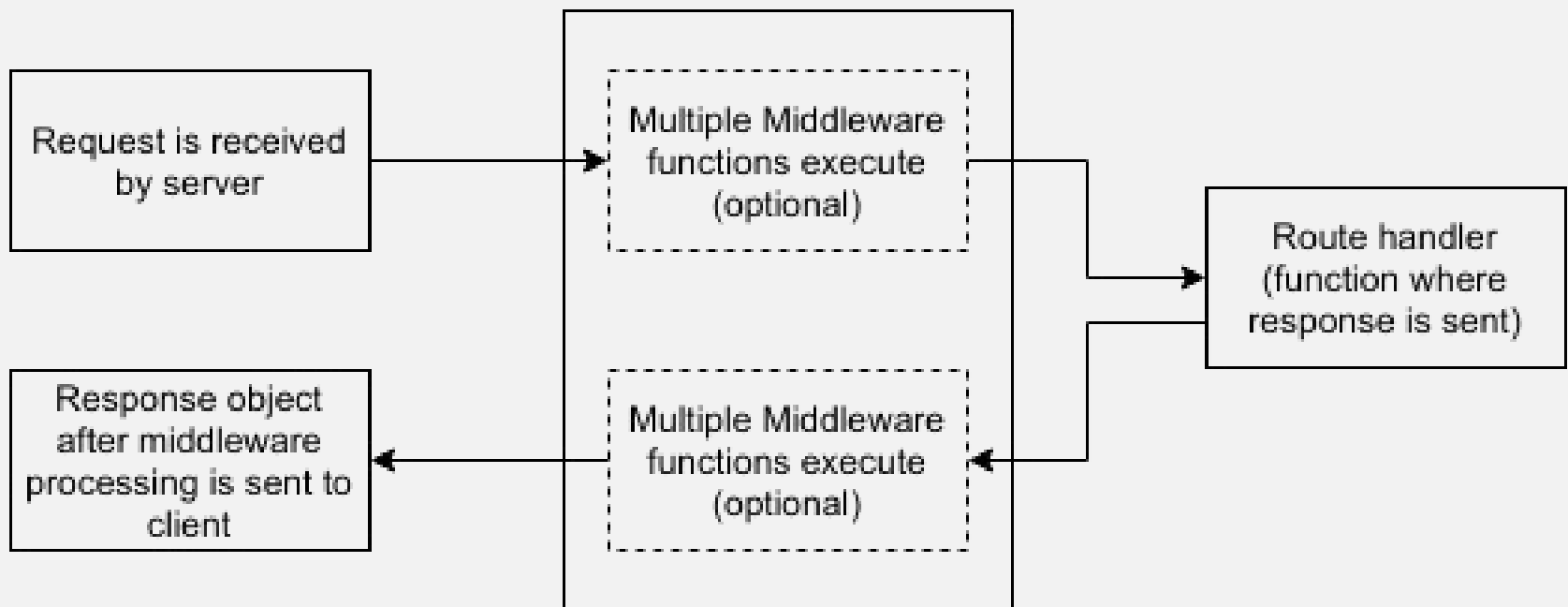
As funções de middleware podem realizar as seguintes tarefas

- Executar qualquer código
- Modificar os objetos de pedido (req) e resposta (res)
- Terminar o ciclo de pedido-resposta
- Chamar a próxima função de *middleware* que está na stack

Se a atual função de *middleware* não está no fim do ciclo de pedido-resposta, ela deve chamar o método *next()* para passar o controlo para a próxima função de *middleware*!

Middleware

Este diagrama resume o “workflow” do middleware



Middleware

Uma aplicação em Express pode utilizar os seguintes tipos de middleware:

- Middleware ao nível da aplicação
- Middleware ao nível do routing
- Middleware para lidar com os erros
- Middleware built-in
- Middleware de terceiros (third-party)

Middleware

Middleware ao nível da aplicação em Express

- *Binding* utilizando as funções `app.use()` e `app.METHOD()`
- A função é executada sempre que `app` recebe um pedido

```
var app = express()

app.use(function (req, res, next) {
  console.log('Time:', Date.now())
  next()
})
```

- A função anterior é sempre chamada antes desta

```
app.get('/user/:id', function (req, res, next) {
  res.send('USER')
})
```


Middleware

Middleware ao nível da routing em Express

- Está ligada a uma instância de `express.Router()`

```
var app = express()
var router = express.Router()

// predicate the router with a check and bail out when needed
router.use(function (req, res, next) {
  if (!req.headers['x-auth']) return next('router')
  next()
})

router.get('/', function (req, res) {
  res.send('hello, user!')
})

// use the router and 401 anything falling through
app.use('/admin', router, function (req, res) {
  res.sendStatus(401)
})
```

Middleware

Middleware para lidar com erros em Express

- Têm quatro parâmetros em vez de três
- Adiciona-se um parâmetro para capturar o erro

```
app.use(function (err, req, res, next) {  
  console.error(err.stack)  
  res.status(500).send('Something broke!')  
})
```

Middleware

Built-in middleware em Express

- *express.static* serve para lidar com *assets* estáticos como ficheiros HTML, imagens, etc.
- *express.json* faz o *parsing* de pedidos com conteúdo json
- *express.urlencoded* faz o *parsing* de pedidos com conteúdo URL-encoded

Middleware

Middleware de terceiros em Express

- É utilizado para adicionar funcionalidades a aplicações Express com código de terceiros
- Instalados como módulos de NodeJS que são carregados para a aplicação, ao nível da aplicação ou do router

```
$ npm install cookie-parser
```

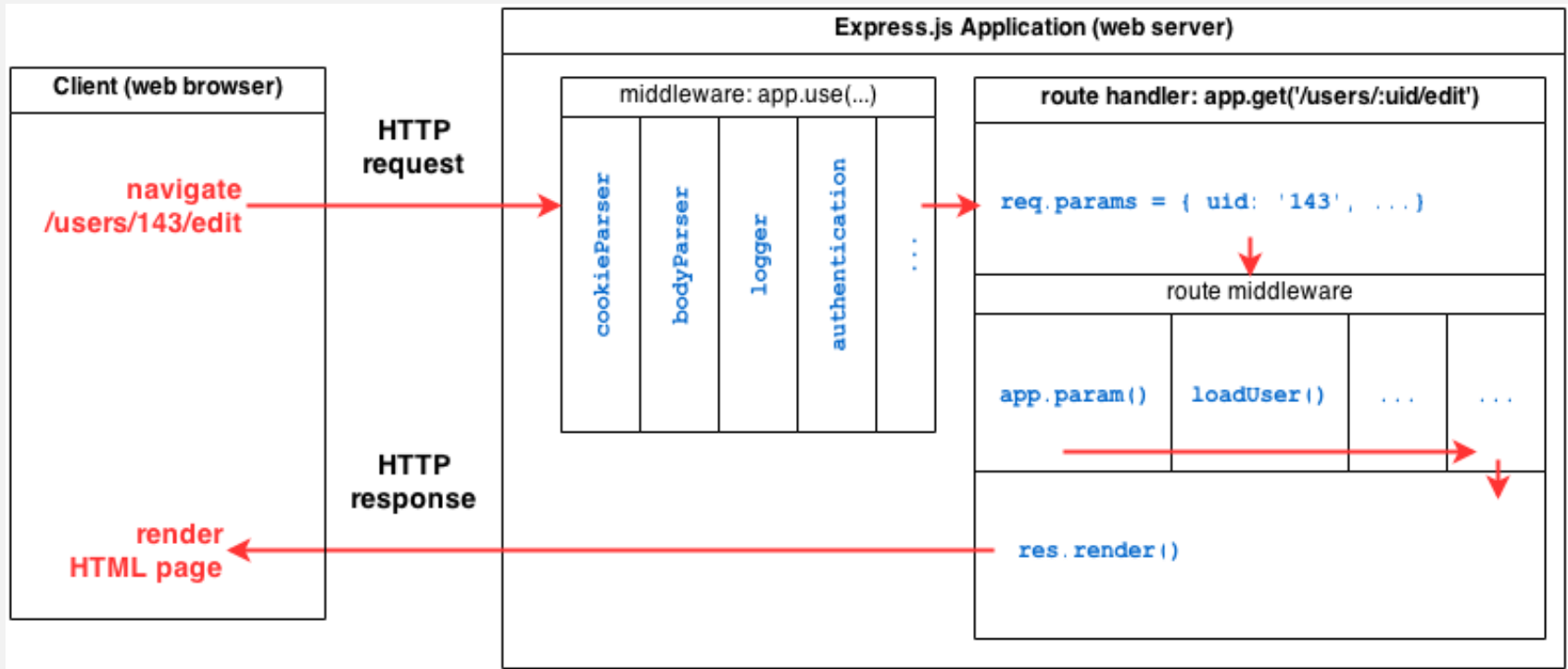
```
var express = require('express')  
var app = express()  
var cookieParser = require('cookie-parser')  
  
// load the cookie-parsing middleware  
app.use(cookieParser())
```

Middleware

Considerações importantes em Express

- A ordem importa
embora o *app.use* seja invocado para cada pedido HTTP, o *middleware* é processado por ordem
- Não esquecer do `next()`
O *routing* do ExpressJS não tem noção do que está a fazer o *middleware*
- Reescrever propriedades
Recorde que os argumentos do pedido e da resposta são os mesmos para funções de *middleware* e *routing*

Middleware



Referências

ExpressJS:

- <https://expressjs.com/>
- https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs

EJS:

- <https://ejs.co/>

P.PORTO

NodeJS Express Middleware

Programação em Ambiente Web